

第12章 故障恢复



第12章 故障恢复

大纲

- 概览
- 故障
- 缓冲区策略
- 预写式日志
- 检查点

■ 事务 Transaction

```
UPDATE savings_accounts  
SET balance = balance - 500  
WHERE account = 3209;
```

Decrement Savings Account 怎么全执

```
UPDATE checking_accounts  
SET balance = balance + 500  
WHERE account = 3208;
```

Increment Checking Account

```
INSERT INTO journal VALUES  
(journal_seq.NEXTVAL, '1B'  
3209, 3208, 500);
```

Record in Transaction Journal

■ DTL (Data Transaction Language) 事务语句

事务启动: BEGIN, START TRANSACTION

事务提交: COMMIT

➤ 将事务对数据库的修改持久地写到数据库中

事务中止: ROLLBACK

➤ 将事务对数据库的修改全部撤销 (undo) , 就像事务从未执行过

➤ 事务既可以被自己中止, 也可以被 DBMS 中止

Atomicity: 原子性

- all or nothing
- 事务的操作要么全部执行, 要么一个也不执行

Consistency: 一致性

- it looks correct to me
- 如果事务程序正确且启动时处于一致状态, 则事务结束时仍处于一致状态

ACID

Isolaton: 隔离性

- as if alone
- 一个事务的执行不受其他事务的干扰

Durability: 持久性

- survive failures
- 事务一旦提交, 它对数据库的修改一定全部持久地写到数据库中

■ 一致性

"It Looks Correct to Me"

事务的一致性由用户保证

➤ 别写错程序

■ 隔离性

"As If Alone"

多个事务的执行有 2 种方式

- 串行执行 $\Rightarrow$ 不破坏隔离性
- 交叉执行 $\Rightarrow$ 可能破坏隔离性

事务的隔离性由 DBMS 保证 (第11章: 并发控制)

- 并发控制: 确定多个事务的操作的正确交叉执行顺序

■ 原子性

"All or Nothing"

事务的执行只有两种结局

- 执行完所有操作后提交 ⇒ 不破坏原子性
- 执行了部分操作后中止 ⇒ 破坏原子性

事务的原子性由 DBMS 保证

- 中止事务执行过的操作必须撤销(undo) (第12章：故障恢复)

■ 持久性

"Survive Failures"

故障导致事务对数据库的修改有 4 种结果

- 提交事务的修改已全部持久存储 ⇒ 不破坏持久性
- 提交事务的修改仅部分持久存储 ⇒ 破坏持久性
- 中止事务的修改有些已持久存储 ⇒ 破坏持久性
- 中止事务的修改未持久存储 ⇒ 不破坏持久性

事务的持久性由 DBMS 保证 (第12章: 故障恢复)

- 重做(redo)提交事务对数据库的修改
- 撤销(undo)中止事务对数据库的修改

■ 故障恢复

故障可能会破坏数据库的一致性

故障恢复。故障发生后，DBMS 将数据库恢复到最新的一致性状态

➤ DEMO

```
CREATE TABLE t (  
    id INT PRIMARY KEY,  
    val INT NOT NULL  
);
```

```
SET autocommit=OFF ;
```

大纲

- 概览
- 故障
- 缓冲区策略
- 预写式日志
- 检查点

■ 故障的类型

- 事务故障
- 系统故障
- 存储介质故障

■ 事务故障

逻辑错误

- 事务由于内部错误而无法完成，如违反完整性约束

内部状态错误

- DBMS 由于内部状态错误(如死锁)而必须中止活跃事务

■ 系统故障

软件故障

- DBMS 实现的 bug 所导致的故障

硬件故障

- 运行 DBMS 的计算机发生崩溃，如断电
- 假设系统崩溃不会损坏非易失存储器中的数据

■ 存储介质故障

- 非易失存储器发生故障，损坏了存储的数据
- 假设数据损坏可以被检测，如使用校验码
- 任何 DBMS 都无法从这种故障中恢复，必须从备份中还原

大纲

- 概览
- 故障
- 缓冲区策略
- 预写式日志
- 检查点

■ Undo vs Redo

DBMS 在故障恢复时会做 2 种操作

- **撤销 (Undo)** : Undo 未完成事务对数据库的修改
- **重做 (Redo)** : Redo 已提交事务对数据库的修改

DBMS 如何支持 undo 和 redo 取决于 DBMS 如何管理缓冲池

- 只需 undo?
- 只需 redo?
- 既要 undo, 又要 redo?
- 既不用 undo, 也不用 redo?

■ 缓冲池策略

Transactions	
T_1	T_2
BEGIN	
$r(A)$	
$A := A + 1$	
$w(A)$	BEGIN
	$r(B)$
	$B := B * 2$
	$w(B)$
	COMMIT
ABORT	

Buffer Pool		
A=2	B=4	C=3

Disk		
A=1	B=2	C=3

- 是否强制(force)在 T_2 提交时将B写回磁盘? ← FORCE策略
- 是否允许在 T_1 提交前覆写磁盘上A的值? ← STEAL策略

■ STEAL 策略

DBMS 是否允许将未提交事务所做的修改写到磁盘并覆盖现有数据?

➤ STEAL: 允许

➤ NO-STEAL: 不允许

■ FORCE 策略

DBMS 是否要求事务在提前必须将其所做的修改全部写回磁盘?

➤ FORCE: 要求

➤ NO-FORCE: 不要求

■ 缓冲池策略

缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE
	I/O效率低	I/O效率高

■ NO-STEAL + FORCE

- NO-STEAL $\implies$ 未提交事务不可能将其修改写回磁盘 $\implies$ 无需undo
- FORCE $\implies$ 已提交事务已将其修改全部写回磁盘 $\implies$ 无需redo

Transactions	
T_1	T_2
BEGIN	
r(A)	
$A := A + 1$	
w(A)	BEGIN
	r(B)
	$B := B * 2$
	w(B)
	COMMIT
ABORT	

Buffer Pool		
A=2	B=4	C=3

Disk		
A=1	B=4	C=3

大纲

- 概览
- 故障
- 缓冲区策略
- 预写式日志
- 检查点

■ 预写式日志 (WAL, Write-Ahead Log)

DBMS 在数据文件之外维护一个日志文件，用于记录事务对数据库的修改

- 假定日志文件存储在稳定存储器中
- 日志记录包含 undo 或 redo 时所需的信息

DBMS 在将修改过的对象写到磁盘之前，必须先将修改此对象的日志记录刷写到磁盘

■ WAL 协议

当事务 T_i 启动时，向日志中写入记录 $\langle \text{tid}, \text{BEGIN} \rangle$

- tid: T_i 的ID (txn ID)

当 T_i 提交时，向日志中写入记录 $\langle \text{tid}, \text{COMMIT} \rangle$

- 在DBMS向应用程序返回确认消息之前，必须保证 T_i 的所有日志记录都已刷写到磁盘

当 T_i 修改对象A时，向日志中写入记录 $\langle \text{tid}, \text{oid}, \text{before}, \text{after} \rangle$

- oid: A的ID (object ID)
- before: A修改前的值(undo时用)
- after: A修改后的值(redo时用)

■ 基于 WAL 的故障恢复

第 1 部分：事务正常执行时的行为

- 记录日志
- 按照缓冲池策略将修改过的对象写到磁盘

第 2 部分：故障恢复时的行为

- 根据日志和缓冲池策略，对事务进行 undo 或 redo

■ 事务的分类

根据日志将事务分为 3 类

已提交事务 (Committed Txn)

- 既有 $\langle T, \text{BEGIN} \rangle$ ，又有 $\langle T, \text{COMMIT} \rangle$

不完整事务 (Incomplete Txn)

- 只有 $\langle T, \text{BEGIN} \rangle$ ，而没有 $\langle T, \text{COMMIT} \rangle$

已中止事务 (Aborted Txn)

- 既有 $\langle T, \text{BEGIN} \rangle$ ，又有 $\langle T, \text{ABORT} \rangle$
- 在事务正常执行和故障恢复过程中，如果 T 所做的修改已全部撤销，则将日志记录 $\langle T, \text{ABORT} \rangle$ 写到日志
- 已中止事务相当于从未执行过，故不需要 undo，更不需要 redo

■ 故障恢复时的行为

已提交事务 (Committed Txn)

- 如果一个已提交事务的修改已全部写到磁盘，则无需 redo
- 否则，redo

不完整事务 (Incomplete Txn)

- 如果一个不完整事务的任何修改都未写到磁盘，则无需 undo
- 否则，undo

缓冲池策略决定了上述行为

■ WAL 协议的分类

根据缓冲池策略的不同，可以实现 3 类 WAL 协议

- Undo Logging: WAL + STEAL + FORCE
- Redo Logging: WAL + NO-STEAL + NO-FORCE
- Redo+Undo Logging: WAL + STEAL + NO-FORCE

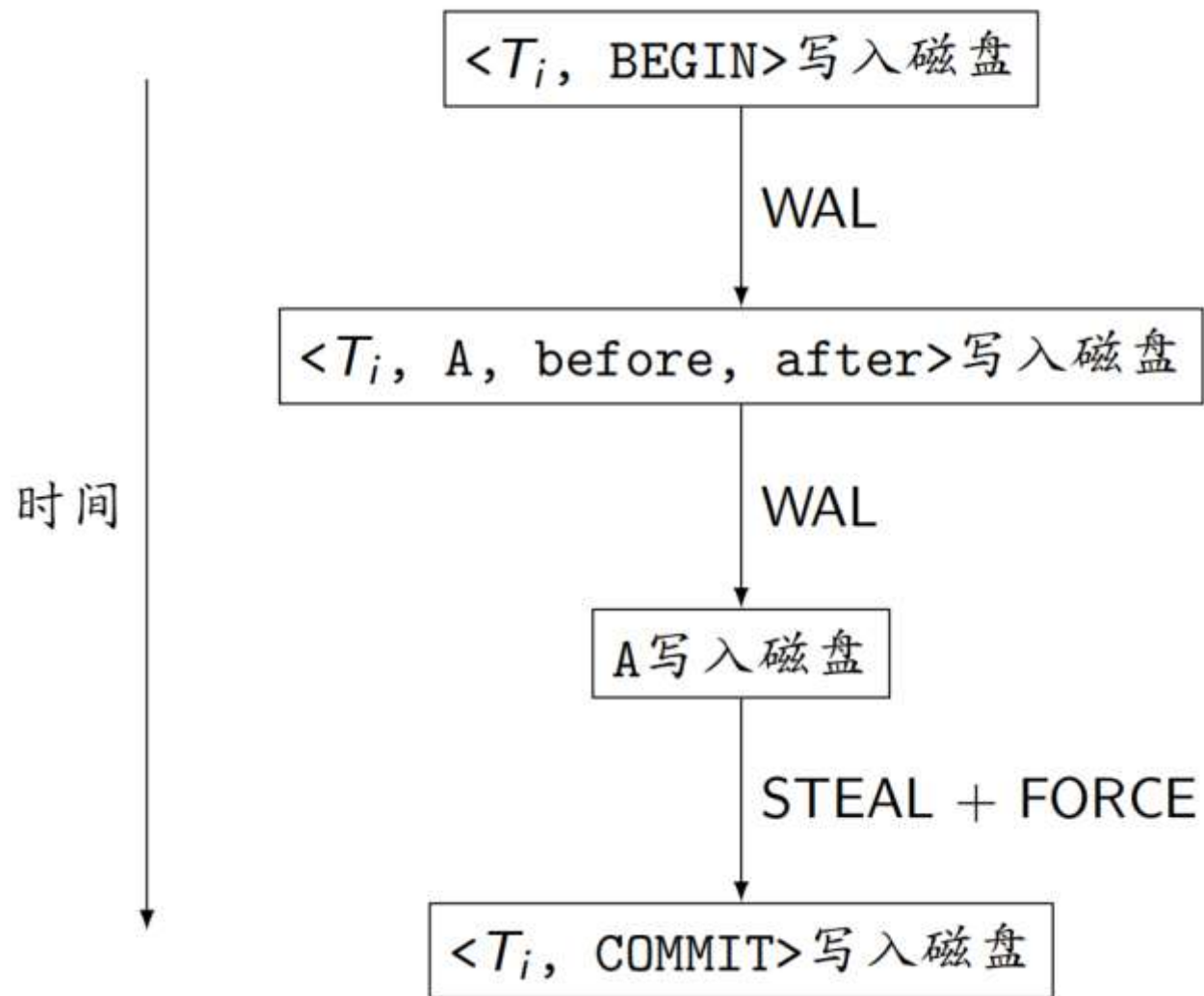
Undo 日志

■ Undo 日志

Undo Logging = WAL + STEAL + FORCE

缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE
	I/O效率低	I/O效率高

■ 基于 Undo 日志的事务正常执行时的行为



■ 基于 Undo 日志的事务正常执行时的行为

Example (Undo Logging)

Step	Action	<i>temp</i> t	<i>内存</i> M_A	M_B	<i>磁盘</i> D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	FLUSH LOG						
9	OUTPUT(A)	16	16	16	16	8	
10	OUTPUT(B)	16	16	16	16	16	
11	COMMIT						<T, COMMIT>
12	FLUSH LOG						

■ 基于 Undo 日志的故障恢复

已提交事务：不需要恢复

➤ FORCE $\Rightarrow$ 已提交事务所做的修改已全部写入磁盘

不完整事务：全部 undo

➤ STEAL $\Rightarrow$ 不完整事务所做的一部分修改可能已经写入磁盘

■ 基于 Undo 日志的故障恢复方法

从后（最后一条记录）向前（第一条记录）扫描日志

根据每条白志记录的类型执行相应的动作

- $\langle T, \text{COMMIT} \rangle$ ：将 T 记录为已提交事务（无需 redo）
- $\langle T, \text{ABORT} \rangle$ ：将 T 记录为已中止事务（无需 undo）
- $\langle T, A, \text{before}, \text{after} \rangle$ ：如果 T 是不完整事务，则将磁盘上 A 的值恢复为 before
- $\langle T, \text{BEGIN} \rangle$ ：T 恢复完毕；如果 T 是不完整事务，则向日志中写入 $\langle T, \text{ABORT} \rangle$
(今后故障恢复时无需再 undo)

■ 基于 Undo 日志的故障恢复

Example (基于Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	FLUSH LOG						
9	OUTPUT(A)	16	16	16	16	8	
10	OUTPUT(B)	16	16	16	16	16	
11	COMMIT						<T, COMMIT>
12	FLUSH LOG						
	Crash!						

■ 基于 Undo 日志的故障恢复

Example (基于Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	FLUSH LOG						
9	OUTPUT(A)	16	16	16	16	8	
10	OUTPUT(B)	16	16	16	16	16	
	Crash!						

■ 基于 Undo 日志的故障恢复

Example (基于Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							$\langle T, \text{BEGIN} \rangle$
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	$\langle T, A, 8, 16 \rangle$
5	READ(B, t)	8	16	8	8	8	
	<i>Crash!</i>						

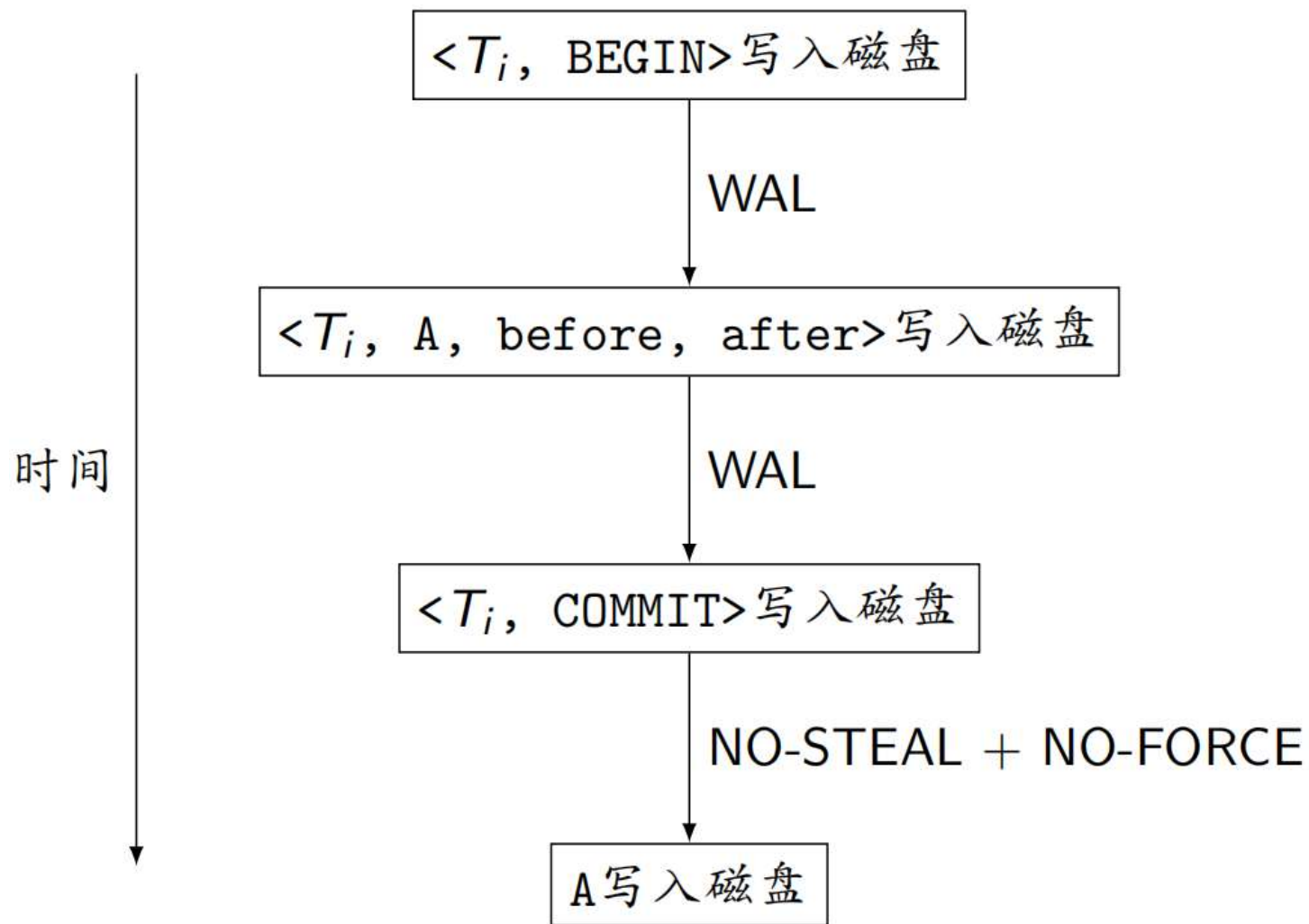
Redo 日志

■ Redo 日志

Redo Logging = WAL + NO-STEAL + NO-FORCE

缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE
	I/O效率低	I/O效率高

■ 事务正常执行时的行为



■ 基于 Redo 日志的事务正常执行时的行为

Example (Redo Logging)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	COMMIT						<T, COMMIT>
9	FLUSH LOG						
10	OUTPUT(A)	16	16	16	16	8	
11	OUTPUT(B)	16	16	16	16	16	

■ 基于Redo 日志的故障恢复

已提交事务：全部 redo

- NO-FORCE $\Rightarrow$ 已提交事务所做的修改可能尚未全部写入磁盘

不完整事务：不需要恢复

- NO-STEAL $\Rightarrow$ 不完整事务所做的任何修改都未写入磁盘

■ 基于 Redo 日志的故障恢复方法

从前（第一条记录）向后（最后一条记录）扫描日志 2 遍

第 1 遍扫描：记录已提交事务和已中止事务

- $\langle T, \text{COMMIT} \rangle$ ：将 T 记录为已提交事务（需要 redo）
- $\langle T, \text{ABORT} \rangle$ ：将 T 记录为已中止事务（无需 undo）

第 2 遍扫描：根据每条日志记录的类型执行相应的动作

- $\langle T, A, \text{before}, \text{after} \rangle$ ：如果 T 是已提交事务，则将磁盘上 A 的值覆写为 after
- $\langle T, \text{BEGIN} \rangle$ ：如果 T 是不完整事务，则向日志中写入 $\langle T, \text{ABORT} \rangle$

■ 基于 Redo 日志的故障恢复

Example (基于Redo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	COMMIT						<T, COMMIT>
9	FLUSH LOG						
10	OUTPUT(A)	16	16	16	16	8	
11	OUTPUT(B)	16	16	16	16	16	
	Crash!						

■ 基于 Redo 日志的故障恢复

Example (基于Redo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	COMMIT						<T, COMMIT>
9	FLUSH LOG						
	<i>Crash!</i>						

■ 基于 Redo 日志的故障恢复

Example (基于Redo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
	Crash!						

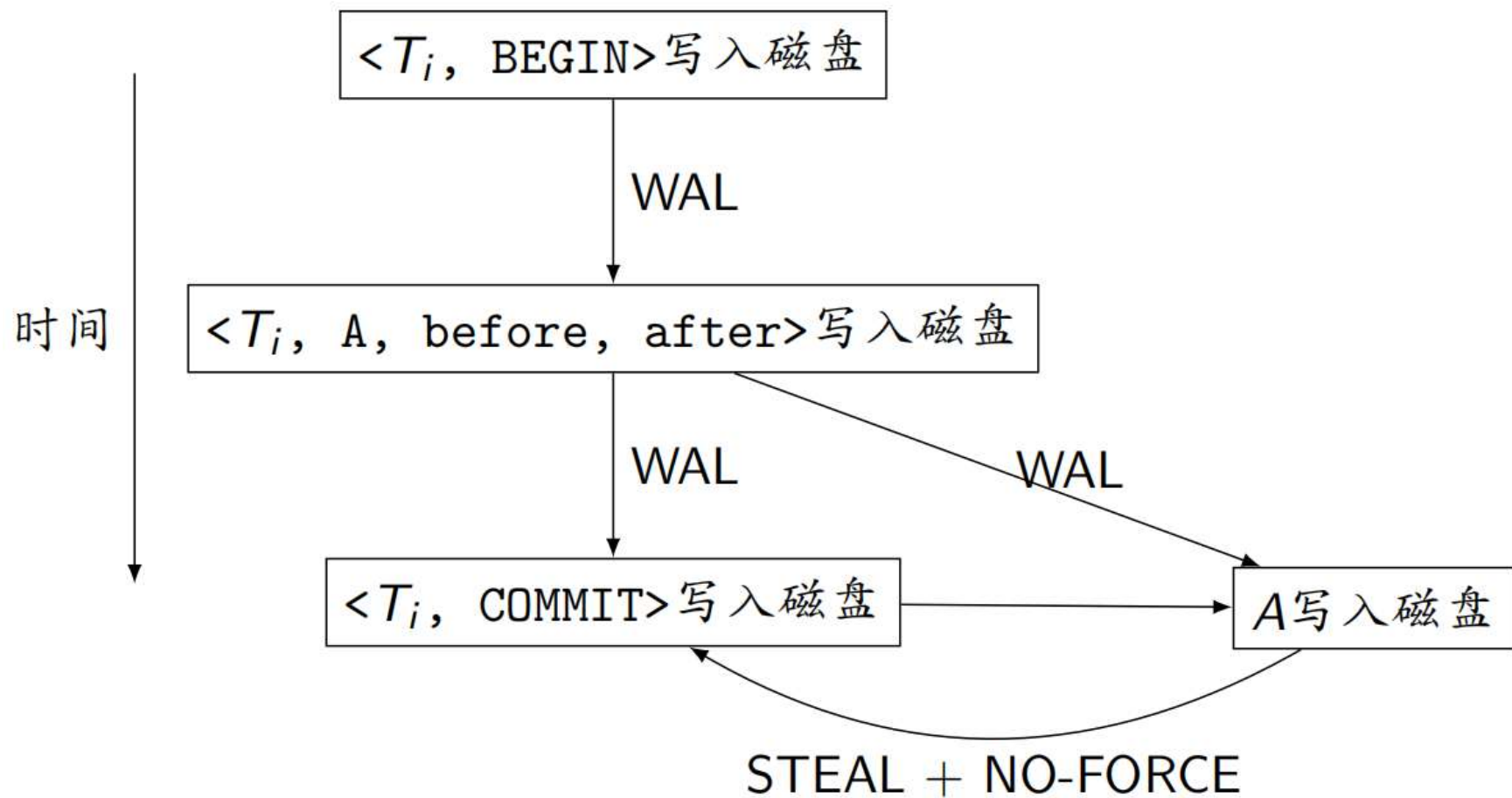
Redo + Undo 日志

■ Redo+Undo 日志

Redo+Undo Logging = WAL + STEAL + NO-FORCE

缓冲池效率高	STEAL + FORCE	STEAL + NO-FORCE
缓冲池效率低	NO-STEAL + FORCE	NO-STEAL + NO-FORCE
	I/O效率低	I/O效率高

■ 基于 Redo + Undo 日志的事务正常执行时的行为



■ 基于 Redo + Undo 日志的事务正常执行时的行为

Example (Redo+Undo Logging)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	t := t * 2	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	t := t * 2	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	OUTPUT(A)	16	16	16	16	8	
9	COMMIT						<T, COMMIT>
10	FLUSH LOG						
11	OUTPUT(B)	16	16	16	16	16	

■ 基于 Redo + Undo 日志的故障恢复

已提交事务：全部 redo

➤ NO-FORCE $\Rightarrow$ 已提交事务所做的修改可能尚未全部写入磁盘

不完整事务：全部 undo

➤ STEAL $\Rightarrow$ 不完整事务所做的一部分修改可能已经写入磁盘

■ 基于 Redo + Undo 日志的故障恢复方法

Redo 阶段：redo 已提交事务

➤ 与基于 Redo 日志的故障恢复方法相同

Undo 阶段：undo 不完整事务

➤ 与基于 Undo 日志的故障恢复方法相同

先 redo，再 undo

■ 基于 Redo + Undo 日志的故障恢复

Example (基于Redo+Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	OUTPUT(A)	16	16	16	16	8	
9	COMMIT						<T, COMMIT>
10	FLUSH LOG						
11	OUTPUT(B)	16	16	16	16	16	
	Crash!						

■ 基于 Redo + Undo 日志的故障恢复

Example (基于Redo+Undo Logging的故障恢复)

Step	Action	t	M_A	M_B	D_A	D_B	Log
1							<T, BEGIN>
2	READ(A, t)	8	8		8	8	
3	$t := t * 2$	16	8		8	8	
4	WRITE(A, t)	16	16		8	8	<T, A, 8, 16>
5	READ(B, t)	8	16	8	8	8	
6	$t := t * 2$	16	16	8	8	8	
7	WRITE(B, t)	16	16	16	8	8	<T, B, 8, 16>
8	OUTPUT(A)	16	16	16	16	8	
	Crash!						

■ 缓冲池策略的比较

运行时效率

	FORCE	NO-FORCE
STEAL	—	Fastest
NO-STEAL	Slowest	—

故障恢复效率

	FORCE	NO-FORCE
STEAL	—	Slowest
NO-STEAL	Fastest	—

几乎所有DBMS都采用**STEAL + NO-FORCE**

■ 组提交

每条日志记录单独刷写到磁盘的 I/O 开销太大

在内存中设置日志缓冲区，将日志记录写到日志缓冲区，然后成批刷写到日志文件

- 日志缓冲区满时刷写
- 定时刷写

大纲

- 概览
- 故障
- 缓冲区策略
- 预写式日志
- 检查点

■ WAL 的问题

- 日志永远在变大
- 故障恢复时需要扫描日志，恢复时间越来越长

Example (WAL)

```
<T1, BEGIN>  
<T1, A, 5, 15>  
<T2, BEGIN>  
<T2, B, 10, 20>  
<T2, C, 15, 25>  
<T2, COMMIT>  
<T3, BEGIN>  
<T1, D, 20, 30>  
<T3, E, 25, 35>  
<T1, COMMIT>  
<T3, F, 30, 40>
```

如果使用Undo Logging，则扫描到这里即可

■ 检查点

DBMS 定期设置检查点

- 将日志刷写到磁盘
- 根据缓冲池策略，将脏页写到磁盘
- 故障恢复时只需扫描到最新的检查点

■ 模糊检查点

检查点开始: 向日志中写入<BEGIN CHECKPOINT ($T_1, T_2, \dots, T_n$)>

- $T_1, T_2, \dots, T_n$ 是检查点开始时的活跃事务(active txn)
- 活跃事务是尚未提交或中止的事务

检查点中间: 根据缓冲池策略, 将脏页(dirty page)写到磁盘

- 如果采用STEAL, 则将全部脏页写到磁盘
- 否则, 只将已提交事务所做的修改写到磁盘

检查点结束: 向日志中写入<END CHECKPOINT>, 并将日志刷写到磁盘

- 如果采用NO-FORCE, 则写完全部脏页后即可结束检查点
- 否则, 在 $T_1, T_2, \dots, T_n$ 全部提交后, 才能结束检查点

■ 涉及检查点的故障恢复

缓冲池策略：STEAL + NO-FORCE

先 redo，再 undo

Redo 阶段：redo 已提交事务

- 从前向后扫描日志
- 从哪条日志记录开始？

Undo 阶段：undo 不完整事务

- 从后向前扫描日志
- 到哪条日志记录为止？

■ Redo 阶段

日志中最新的完整检查点

<BEGIN CHECKPOINT ($T_1, T_2, \dots, T_n$)>

...

<END CHECKPOINT>

需要 redo 的最早的事务一定属于 $\{T_1, T_2, \dots, T_n\}$

从日志记录 <BEGIN CHECKPOINT ($T_1, T_2, \dots, T_n$)> 开始向后扫描日志

➤ 不需要从最早的 < T_i , BEGIN> 开始扫描

■ 证明 I

需要redo的最早的事务 T 一定属于 $\{T_1, T_2, \dots, T_n\}$

Log	Fact
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle T, \text{COMMIT} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \notin \{T_1, T_2, \dots, T_n\}$
...	
$\langle \text{END CHECKPOINT} \rangle$	T 所做的修改已全部写到磁盘，无需redo

■ 证明 II

Log	Fact
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \in \{T_1, T_2, \dots, T_n\}$
...	
$\langle T, \text{COMMIT} \rangle$	
...	
$\langle \text{END CHECKPOINT} \rangle$	T 所做的修改已全部写到磁盘，无需redo

■ 证明 III

Log	Fact
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \in \{T_1, T_2, \dots, T_n\}$
...	
$\langle \text{END CHECKPOINT} \rangle$	
...	
$\langle T, \text{COMMIT} \rangle$	T 所做的修改未必全部 写到磁盘，必须redo

■ Undo 阶段

日志中最新的完整检查点

$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$

...

$\langle \text{END CHECKPOINT} \rangle$

需要 undo 的最早的事务一定属于 $\{T_1, T_2, \dots, T_n\}$

扫描到 $T_1, T_2, \dots, T_n$ 中最早的事务 T_i 的日志记录 $\langle T_i, \text{BEGIN} \rangle$ 为止

■ 证明

需要undo的最早的事务 T 一定属于 $\{T_1, T_2, \dots, T_n\}$

Log	Fact
$\langle T, \text{BEGIN} \rangle$	
...	
$\langle \text{BEGIN CHECKPOINT } (T_1, T_2, \dots, T_n) \rangle$	$T \in \{T_1, T_2, \dots, T_n\}$
...	
$\langle \text{END CHECKPOINT} \rangle$	T 所做的部分修改可能 已写到磁盘，必须undo

■ 涉及检查点的故障恢复-Redo 阶段

缓冲池策略: STEAL + NO-FORCE

Example (涉及检查点的故障恢复—Redo阶段)

Log	Redo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始redo
$\langle T_2, C, 15, 25 \rangle$	
$\langle T_3, D, 20, 30 \rangle$	
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	

■ 涉及检查点的故障恢复-Redo 阶段

缓冲池策略: STEAL + NO-FORCE

Example (涉及检查点的故障恢复—Redo阶段)

Log	Redo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始redo
$\langle T_2, C, 15, 25 \rangle$	$C \leftarrow 25$
$\langle T_3, D, 20, 30 \rangle$	
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	

■ 涉及检查点的故障恢复-Redo 阶段

缓冲池策略: STEAL + NO-FORCE

Example (涉及检查点的故障恢复—Redo阶段)

Log	Redo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始redo
$\langle T_2, C, 15, 25 \rangle$	$C \leftarrow 25$
$\langle T_3, D, 20, 30 \rangle$	T_3 无需redo
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	

■ 涉及检查点的故障恢复-Redo 阶段

缓冲池策略: STEAL + NO-FORCE

Example (涉及检查点的故障恢复—Redo阶段)

Log	Redo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	从这里开始redo
$\langle T_2, C, 15, 25 \rangle$	$C \leftarrow 25$
$\langle T_3, D, 20, 30 \rangle$	T_3 无需redo
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 redo完毕

■ 涉及检查点的故障恢复-Undo 阶段

缓冲池策略: STEAL + NO-FORCE

Example (涉及检查点的故障恢复—Undo阶段)

Log	Undo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	
$\langle T_2, C, 15, 25 \rangle$	
$\langle T_3, D, 20, 30 \rangle$	
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 无需 undo

■ 涉及检查点的故障恢复-Undo 阶段

缓冲池策略: STEAL + NO-FORCE

Example (涉及检查点的故障恢复—Undo阶段)

Log	Undo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	
$\langle T_2, C, 15, 25 \rangle$	
$\langle T_3, D, 20, 30 \rangle$	$D \leftarrow 20$
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 无需undo

■ 涉及检查点的故障恢复-Undo 阶段

缓冲池策略: STEAL + NO-FORCE

Example (涉及检查点的故障恢复—Undo阶段)

Log	Undo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	
$\langle T_2, C, 15, 25 \rangle$	T_2 无需undo
$\langle T_3, D, 20, 30 \rangle$	$D \leftarrow 20$
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 无需undo

■ 涉及检查点的故障恢复-Undo 阶段

缓冲池策略: STEAL + NO-FORCE

Example (涉及检查点的故障恢复—Undo阶段)

Log	Undo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	
$\langle T_3, B, 10, 20 \rangle$	$B \leftarrow 10$
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	
$\langle T_2, C, 15, 25 \rangle$	T_2 无需undo
$\langle T_3, D, 20, 30 \rangle$	$D \leftarrow 20$
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 无需undo

■ 涉及检查点的故障恢复-Undo 阶段

缓冲池策略: STEAL + NO-FORCE

Example (涉及检查点的故障恢复—Undo阶段)

Log	Undo Action
$\langle T_1, \text{BEGIN} \rangle$	
$\langle T_1, A, 5, 15 \rangle$	
$\langle T_2, \text{BEGIN} \rangle$	
$\langle T_1, \text{COMMIT} \rangle$	
$\langle T_3, \text{BEGIN} \rangle$	T_3 undo完毕, 写 $\langle T_3, \text{ABORT} \rangle$
$\langle T_3, B, 10, 20 \rangle$	$B \leftarrow 10$
$\langle \text{BEGIN CHECKPOINT } (T_2, T_3) \rangle$	
$\langle T_2, C, 15, 25 \rangle$	T_2 无需undo
$\langle T_3, D, 20, 30 \rangle$	$D \leftarrow 20$
$\langle \text{END CHECKPOINT} \rangle$	
$\langle T_2, \text{COMMIT} \rangle$	T_2 无需undo

大纲

- 概览
- 故障
- 缓冲区策略
- 预写式日志
- 检查点